Praktikum 4 Inheritance



Nama : Muhamad Haikal Dwi Arya
Hari/Tanggal : Senin, 06/10/2026
Kelas : D3 – 2A
NIM : 251511018

Politeknik Negeri Bandung
Jurusan Teknik Komputer dan Informatika

Bagian A - Circle dan Cylinder

1. Code Inti

```
// Circle.java (Superclass)
public class Circle {
    private double radius;
    private String color;

    public Circle() { radius = 1.0; color = "red"; }
    public Circle(double radius) { this.radius = radius; color = "red"; }
    public Circle(double radius, String color) { this.radius = radius; this.color = color; }
    public double getRadius() { return radius; }
    public void setRadius(double radius) { this.radius = radius; }
    public double getArea() { return radius * radius * Math.PI; }
    public String getColor() { return color; }
    public void setColor(String color) { this.color = color; }
    @Override
    public String toString() { return "Circle[radius=" + radius + ", color=" + color + "]"; }
}
// Cylinder.java (Subclass extends Circle)
public class Cylinder extends Circle {
    private double height;

    public Cylinder() { super(); this.height = 1.0; }
    public Cylinder(double height) { super(); this.height = height; }
    public Cylinder(double radius, double height) { super(radius); this.height = height; }
    public double getHeight() { return height; }
    public void setHeight(double height) { this.height = height; }
    @Override
    public double getArea() {
        return 2 * Math.PI * getRadius() * height + 2 * Math.PI * getRadius() * getRadius();
    }
    public double getVolume() { return super.getArea() * height; }
    @Override
    public String toString() {
        return "Cylinder: subclass of " + super.toString() + ", height=" + height;
    }
}
```

2. Output TestCylinder (Step 5)

```
$ java -cp . TestCylinder
c1
radius = 1.0
height = 1.0
area = 12.566370614359172
volume = 3.141592653589793

c2
radius = 1.0
height = 10.0
area = 69.11503837897544
volume = 31.41592653589793

c3
radius = 2.0
height = 10.0
area = 150.79644737231007
volume = 125.66370614359172

Pengujian toString() Cylinder (Step 9):
Cylinder: subclass of Circle[radius=2.0, Shape[color=green, filled=true]], height=10.0
```

Menguji pewarisan method getRadius() dan getArea() pada Cylinder. Terbukti subclass dapat langsung memanggil method publik superclass.

3. Bukti Constructor Chain (Step 6)

```
$ javac *.java; java TestCylinder
Circle(double) dipanggil
Cylinder(double,double) dipanggil
c1
radius = 1.0
height = 1.0
area = 12.566370614359172
volume = 3.141592653589793

c2
radius = 1.0
height = 10.0
area = 69.11503837897544
```

Mengamati urutan konstruktor. Terbukti konstruktor Circle dieksekusi lebih dahulu sebelum konstruktor Cylinder.

4. Bukti Akses Private Ditolak (Step 7)

```
LENOVO@LAPTOP-CTR1HPRD MINGW64 ~/Downloads/s3/PBO NEW/INHERITANCE, GE
tikum4a/src
$ javac Cylinder.java
Cylinder.java:41: error: radius has private access in Circle
    return radius; // radius adalah field private di Circle
           ^
1 error
```

Menguji batas enkapsulasi dengan mengakses variabel private radius. Terjadi compile error karena subclass tidak dapat mengakses langsung field private superclass.

5. Override getArea() dan Bug getVolume() (Step 8)

```
c3
radius = 2.0
height = 10.0
area = 150.79644737231007
volume = 1507.9644737231006

Pengujian toString() Cylinder (Step 9):
Cylinder: subclass of Circle[radius=2.0, Shape[color=green, filled=true]], height=10.0
```

Menguji overriding getArea(). Nilai getVolume() menjadi salah karena mengalikan tinggi dengan luas permukaan silinder, bukan luas alas.

6. Perbaikan super.getArea() pada getVolume()

```
c3
radius = 2.0
height = 10.0
area = 150.79644737231007
volume = 125.66370614359172

Pengujian toString() Cylinder (Step 9):
Cylinder: subclass of Circle[radius=2.0, Shape[color=green, filled=true]], height=10.0
```

Menguji perbaikan getVolume() menggunakan super.getArea() * height. Hasil volume kembali benar karena secara eksplisit memanggil implementasi milik superclass.

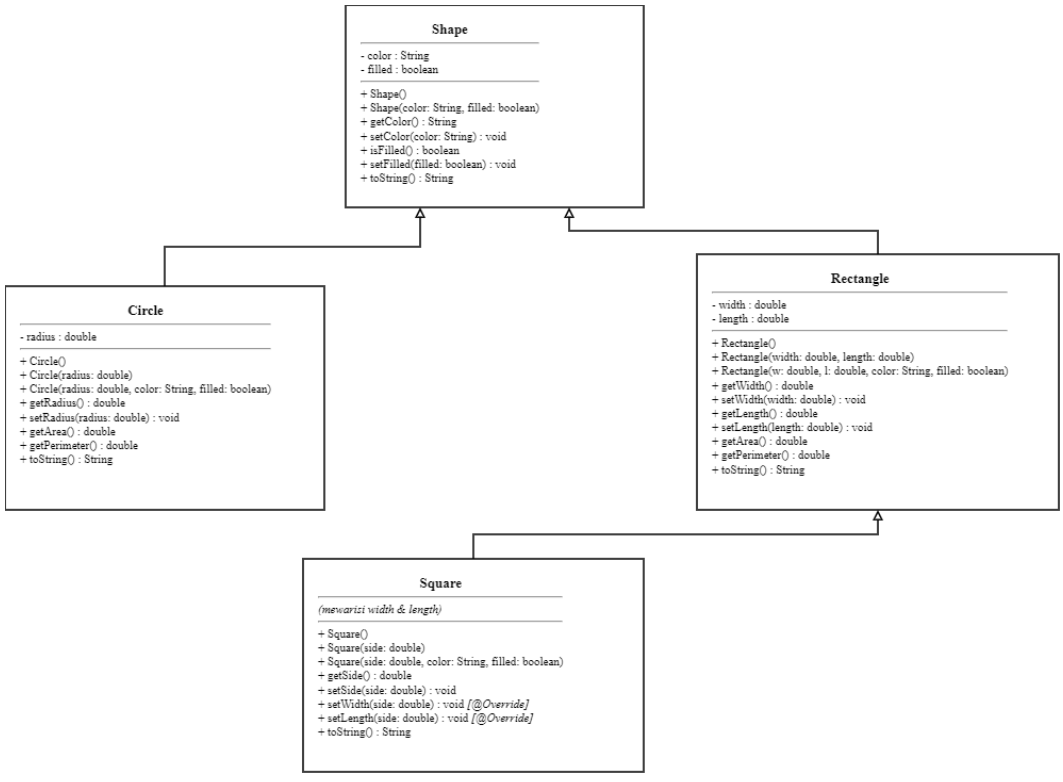
7. Override toString() dan Uji Anotasi @Override (Step 9)

```
LENOVO@LAPTOP-CTR1HPRD MINGW64 ~/Downloads/s3/PBO NEW/INHERITANCE, GENERALIZATION & OVERRID
tikum4a/src
$ javac Cylinder.java
Cylinder.java:36: error: method does not override or implement a method from a supertype
    @Override
    ^
1 error
```

Menguji validasi compiler. Terjadi error saat nama method sengaja diubah menjadi toStrng(), membuktikan anotasi @Override mencegah kesalahan penulisan nama method.

Bagian B - Shape Hierarchy

1. Diagram Hierarki Shape



2. Code Inti Shape Hierarchy

```
// Shape.java
public class Shape {
    private String color;
    private boolean filled;
    public Shape() { color = "green"; filled = true; }
    public Shape(String color, boolean filled) { this.color = color; this.filled =
filled; }
    public String getColor() { return color; }
    public void setColor(String color) { this.color = color; }
    public boolean isFilled() { return filled; }
    public void setFilled(boolean filled) { this.filled = filled; }
    @Override
    public String toString() { return "Shape[color=" + color + ", filled=" + filled +
"]"; }
}

// Rectangle.java (extends Shape)
public class Rectangle extends Shape {
    private double width, length;
    public Rectangle() { super(); width = 1.0; length = 1.0; }
    public Rectangle(double width, double length) { super(); this.width = width;
this.length = length; }
    public Rectangle(double width, double length, String color, boolean filled) {
        super(color, filled); this.width = width; this.length = length;
    }
    public double getWidth() { return width; }
    public void setWidth(double width) { this.width = width; }
    public double getLength() { return length; }
    public void setLength(double length) { this.length = length; }
    public double getArea() { return width * length; }
    public double getPerimeter() { return 2 * (width + length); }
    @Override
    public String toString() {
        return "Rectangle[width=" + width + ", length=" + length + ", " +
super.toString() + "]";
    }
}

// Square.java (extends Rectangle)
public class Square extends Rectangle {
    public Square() { super(1.0, 1.0); }
    public Square(double side) { super(side, side); }
    public Square(double side, String color, boolean filled) { super(side, side, color,
filled); }
    public double getSide() { return getWidth(); }
    public void setSide(double side) { super.setWidth(side); super.setLength(side); }
    @Override
    public void setWidth(double side) { super.setWidth(side); super.setLength(side); }
    @Override
    public void setLength(double side) { super.setWidth(side); super.setLength(side); }
    @Override
    public String toString() { return "Square[side=" + getSide() + ", " +
super.toString() + "]"; }
}
```

3. Output Pengujian TestShape (Step 16)

```
LENOVO@LAPTOP-CTR1HPRD MINGW64 ~/Downloads/s3/PBO NEW/INHERITANCE, GENERALIZATION & OVER
RIDING/praktikum4a/src
$ javac *.java
java -cp . TestShape
Shape[color=black, filled=false]
Circle[radius=2.0, Shape[color=blue, filled=true]] area=12.566370614359172
Rectangle[width=3.0, length=4.0, Shape[color=yellow, filled=true]] area=12.0
Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]] area
=25.0
```

Menguji hierarki class Shape, Circle, Rectangle, dan Square. Seluruh objek berhasil mencetak representasi teks dan luas areanya masing-masing.

4. Bukti Square Sebelum dan Sesudah Override Setter (Step 14 & 15)

```
LENOVO@LAPTOP-CTR1HPRD MINGW64 ~/Downloads/s3/PBO NEW/INHERITANCE, GENERALIZATION & OVERRIDING/praktikum4a/src
$ javac *.java
java -cp . TestShape
awal : Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]]
area : 25.0
sesudah setWidth(8): Square[side=8.0, Rectangle[width=8.0, length=8.0, Shape[color=green, filled=true]]]
sesudah setLength(3): Square[side=3.0, Rectangle[width=3.0, length=3.0, Shape[color=green, filled=true]]]
```

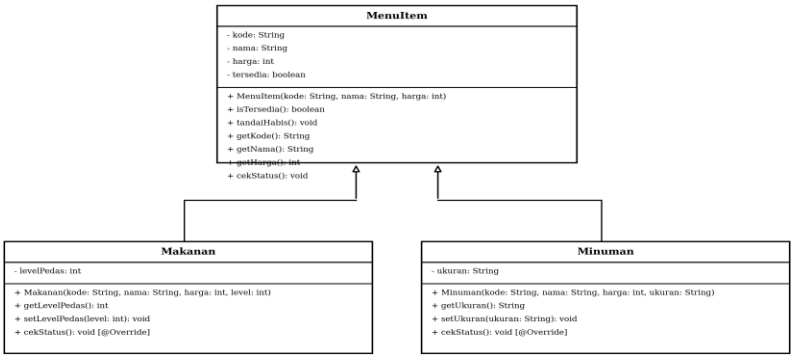
Menguji konsistensi bujur sangkar. Setter bawaan Rectangle membuat panjang dan lebar berbeda, sedangkan setter yang telah dioverride menjaga panjang dan lebar selalu sama.

Bagian C - Kasus Tugas Sebelumnya

1. Alasan Memilih Inheritance

Memenuhi relasi IS-A: Makanan dan Minuman adalah MenuItem. Keduanya mewarisi atribut kode, nama, harga, dan ketersediaan, serta memiliki atribut spesifik masing-masing (level pedas dan ukuran).

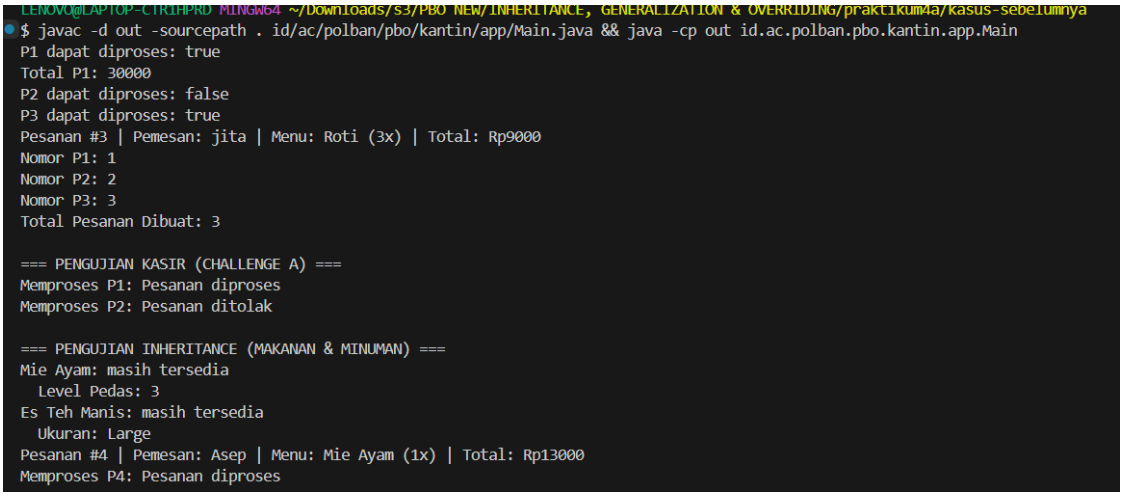
2. Diagram Hierarki Kasus Kantin



3. Code Extends, super(...), @Override, dan super.method(...)

```
// Makanan.java
package id.ac.polban.pbo.kantin.model;
public class Makanan extends MenuItem {
    private int levelPedas;
    public Makanan(String kode, String nama, int harga, int levelPedas) {
        super(kode, nama, harga);
        this.levelPedas = levelPedas;
    }
    public int getLevelPedas() { return levelPedas; }
    public void setLevelPedas(int levelPedas) { this.levelPedas = levelPedas; }
    @Override
    public void cekStatus() {
        super.cekStatus();
        System.out.println("  Level Pedas: " + levelPedas);
    }
}
// Minuman.java
package id.ac.polban.pbo.kantin.model;
public class Minuman extends MenuItem {
    private String ukuran;
    public Minuman(String kode, String nama, int harga, String ukuran) {
        super(kode, nama, harga);
        this.ukuran = ukuran;
    }
    public String getUkuran() { return ukuran; }
    public void setUkuran(String ukuran) { this.ukuran = ukuran; }
    @Override
    public void cekStatus() {
        super.cekStatus();
        System.out.println("  Ukuran: " + ukuran);
    }
}
```

4. Output Pengujian Main.java (Step 20)



Menguji fitur kantin lama dan subclass baru. Pesanan lama (P1-P3) tetap normal (regresi aman), serta Makanan dan Minuman berhasil menampilkan atribut khususnya.

Tabel Skenario Pengujian (T4-01 s.d. T4-07)

ID	Skenario Uji	Expected Result	Status	Bukti
T4-01	Object subclass dibuat melalui constructor	Bagian superclass dan subclass terinisialisasi benar	**PASS**	new Makanan("F001", "Mie Ayam", 13000, 3) berhasil menginisialisasi state.
T4-02	Method warisan dipanggil dari subclass	Behavior superclass tersedia tanpa copy code	**PASS**	mieAyam.tandaiHabis() memanggil method milik superclass MenuItem.
T4-03	Method override dipanggil	Behavior subclass yang dijalankan	**PASS**	mieAyam.cekStatus() mengeksekusi method override milik subclass.
T4-04	Override memakai super.method(...)	Bagian behavior superclass tetap digunakan	**PASS**	super.cekStatus() berhasil dipanggil di dalam method override.
T4-05	Akses state mengikuti encapsulation	Tidak mengakses field private secara langsung	**PASS**	Subclass membaca field private superclass via getter.
T4-06	Dua subclass diuji	Perbedaan behavior terlihat pada output	**PASS**	Makanan menampilkan level pedas, Minuman menampilkan ukuran.
T4-07	Regression tugas lama	Fungsi utama sebelumnya tetap berjalan	**PASS**	Alur pesanan lama (P1-P3 dan kasir) tetap berjalan normal.

Jawaban Refleksi (Bagian 7.3)

1. Apa yang membuat Circle layak menjadi class yang lebih umum daripada Cylinder?
Jawab: Silinder merupakan lingkaran yang diperluas ke bentuk 3D dengan atribut tinggi, sedangkan lingkaran merupakan konsep dasar 2D.

2. Berdasarkan Step 5, apa arti praktis extends bagi object Cylinder?
Jawab: Objek Cylinder dapat langsung memakai seluruh method publik Circle tanpa perlu menyalin atau menulis ulang kodenya.

3. Mengapa constructor Circle dijalankan ketika Anda membuat object Cylinder?
Jawab: Java harus membangun bagian superclass terlebih dahulu di memori agar state yang diwariskan valid sebelum subclass menambahkan state khususnya.

4. Mengapa getVolume() menjadi salah setelah getArea() dioverride?
Jawab: Karena getVolume() memanggil getArea() yang telah dioverride menjadi rumus luas permukaan silinder, bukan luas alas lingkaran.

5. Apa yang secara eksplisit Anda minta ketika menulis super.getArea() atau super.toString()?
Jawab: Meminta Java mengeksekusi implementasi asli milik superclass (Circle), mengabaikan method override pada subclass.

6. Mengapa Square perlu mengubah setter yang diwarisi dari Rectangle?
Jawab: Untuk menjaga aturan bujur sangkar agar panjang dan lebar selalu sama setiap kali salah satu sisi diubah.

7. Apakah setiap dua class yang mirip harus menggunakan inheritance? Jelaskan dari salah satu kasus Anda.
Jawab: Tidak. Harus memiliki relasi IS-A. Pada kasus kantin, Mahasiswa dan MenuItem sama-sama punya nama dan kode/NIM, tetapi hubungannya adalah asosiasi dalam Pesanan, bukan inheritance.

8. Lengkapi dengan bahasa Anda sendiri: 'Inheritance bukan sekadar mengurangi duplikasi, tetapi ...'
Jawab: 'Inheritance bukan sekadar mengurangi duplikasi, tetapi membentuk relasi hierarki tipe logis agar kode mudah diperluas dan aturan objek tetap konsisten.'

Self Assessment (Bagian 7.2)

Pernyataan Evaluasi Mandiri	Skor (1-5)
Saya dapat menjelaskan extends dari code yang saya jalankan.	**4**
Saya dapat membedakan inherited method dan overridden method.	**4**
Saya memahami mengapa constructor subclass memanggil super(...).	**3**
Saya dapat menggunakan super.method(...) dengan alasan yang tepat.	**4**
Saya dapat menjaga encapsulation pada hierarchy class.	**4**
Saya dapat menentukan kapan inheritance masuk akal pada kasus sendiri.	**3**
Kasus tugas sebelumnya telah direfactor dan lulus skenario uji utama.	**4**